Robust classification of Bangladeshi diseased tomato leaves using deep learning techniques

Shafique Iqbal[1] Vaishnavi M[1] Dhiraj K[1]

[1]Department of Computer Science and Engineering,
Vellore Institute of Technology, Chennai, India

Abstract

The rapid growth of computer vision techniques has enabled automated disease detection systems to support large-scale agriculture, where manual inspection is often inefficient and subjective. This work evaluates multiple deep learning models for classifying healthy and diseased tomato leaves using a comprehensive dataset of tomato leaf images collected from agricultural fields in Bangladesh. Tomato leaf images are preprocessed and used to train a range of classifiers under a unified experimental framework. Model performance is compared using accuracy, precision, recall, and F1-score metrics. The best results were obtained by convolutional neural network and VGG19 models, with accuracies of 98.33% and F1-scores of 0.98. The convolutional neural network model achieved precision and recall of 0.98, and the VGG19 model achieved precision and recall of 0.985. The comparative analysis focuses on identifying models that achieve high accuracy and low error rates while maintaining consistent performance across disease categories. The findings aid the selection of effective classification models for automated tomato leaf disease detection systems and contribute to the development of reliable, scalable agricultural decision-support tools.

Keywords: plant disease classification; image classification; deep learning; CNN; VGG19

1. Introduction

Image classification has undergone tremendous growth in recent years due to concurrent advancements in the processing powers of CPUs and GPUs [1]. This has led to the direct application of neural networks and deep learning (DL) methods in the area of research concerning image classification [2]. Deep learning methods are considered the most efficient performers in image classification [3].

Automated plant leaf disease detection has attracted significant attention due to its importance in improving agricultural productivity and reducing crop losses [4]. Early plant disease diagnosis is critical because diseases usually manifest visible symptoms on leaves, and traditional manual inspection is labour-intensive, subjective, and inefficient [5]. Consequently, researchers have increasingly explored image processing, machine learning (ML), and deep learning models to automatically classify healthy and diseased leaves [6].

Early research efforts in automated plant disease detection primarily employed traditional image processing techniques combined with machine learning algorithms. These systems typically followed a structured pipeline consisting of image acquisition, preprocessing, segmentation of diseased regions, feature extraction, and classification. Handcrafted features based on colour, texture, and shape descriptors were widely used to characterise disease symptoms, and classifiers such as Support Vector Machines (SVM), k-Nearest Neighbours (K-NN), and Artificial Neural Networks (ANN) demonstrated encouraging performance under controlled conditions [7, 8, 9]. However, such approaches are highly dependent on the quality of feature engineering and often exhibit limited robustness to variations in illumination, background clutter, leaf orientation, and disease severity.

Several review studies consolidate these advancements and highlight the growing dominance of deep learning in plant leaf disease detection. Convolutional neural network (CNN) architectures such as VGG, ResNet, EfficientNet, GoogleNet, and MobileNet are consistently reported to achieve the highest accuracies across diverse plant disease datasets. These reviews also emphasise the need for larger real-world datasets, improved generalisation, and explainable artificial intelligence techniques to support reliable agricultural decision-making [4, 10].

This work furthers the development of automated plant disease classification systems by:

- using a recently published, real-time dataset [11]
- comparing 9 architectures across various preprocessing conditions
- following a unified experimental framework

The rest of the manuscript is organised as follows: Section II reviews the relevant literature, Section III details the experimental methodology, Section IV discusses the experimental results, and Section V presents the conclusions, with future scope.

2. Related Works

2.1. Machine learning-based approaches for plant leaf disease classification

[7] classified healthy from diseased tomato leaves and identified the disease (late blight, mosaic virus and Septoria leaf spot) through colour analysis by Grey Level Co-occurrence Matrix (GLCM) and segmentation of the diseased part using the HSV colour spectrum.

After pre-processing and segmentation, the vectors obtained were passed through Multilayer Perceptron (MLP), K-NN and SVM classifiers in parallel, following the decision rule that if any of the two classifiers arrived at the same result, the leaf would be classified according to that result. This model achieved an accuracy of 86.45% in distinguishing healthy from diseased tomato leaves and 97.39% in classifying the type of disease affecting a diseased leaf.

[8] carried out a comparative analysis between the SVM, K-NN and CNN models on a Soyabean dataset. The SVM model yielded an accuracy of 76%. The K-NN implementation followed a similar methodology, but a Histogram of Oriented Gradients approach was used to extract features from the Soyabean images. The Manhattan-distance model yielded an accuracy of 64%. In the CNN model, all images were segmented using K-means clustering, and the "b" channel of the RGB colour format was identified as the most effective for clustering. Subsequently, clustering was performed, the leaf component was identified, and the results were fed into the CNN model for classification. The model achieved 96% accuracy, easily outperforming SVM and K-NN for Soyabean leaf disease detection and classification.

[12] employed an automated Fuzzy Relevance Vector Machine (FRVM) model to identify plant leaf types, achieving an accuracy of 99.87%. The FRVM model utilised fuzzy logic with Relevance Vector Machines to classify leaf types by addressing SVM drawbacks in kernel choice and speed through probabilistic, sparse solutions.

[13] applied image segmentation to isolate infected leaf regions, and passed the results to a multi-class SVM classifier using features such as energy, entropy, correlation, and homogeneity, achieving a classification accuracy of 93.75% for tomato leaf diseases.

[9] developed a multiclass plant leaf disease detection system based on traditional image processing and machine learning techniques, involving image pre-processing, segmentation using Otsu's thresholding, and extraction of multiple handcrafted features, including colour moments, texture features (GLCM), shape, vein, Gabor wavelet, and Zernike moment features. The extracted features were classified using Random Forest, SVM, K-NN and ANN classifiers. Experiments conducted on the PlantVillage dataset [14] demonstrated that the Random Forest classifier outperformed other models, achieving the highest classification accuracy of 73.38%.

2.2. Convolutional neural network-based approaches

In the work carried out in [15], a novel convolutional neural network scheme, which had three convolution and max pooling layers with a varying number of filters in each layer, was applied on the PlantVillage dataset. The model, compared with VGG16, InceptionV3 and MobileNet, had a much higher accuracy of 91.2% while using only 1.5MB of storage space compared to accuracies of 63.4%-77.2% and storage spaces of 82MB-163MB. The authors suggested that overfitting and deep architectures hold back pre-trained models, however fast the model might be.

The impact of image segmentation on the performance of convolutional neural network (CNN) models was explored for tomato plant disease detection in [16]. A proposed segmented CNN (S-CNN) approach, where diseased regions of tomato leaves were segmented before training, was compared with a conventional full-image CNN (F-CNN). Experiments conducted on tomato leaf images from the PlantVillage dataset demonstrated that while the F-CNN achieved high accuracy on similar training data, its performance dropped significantly when tested on independent real-world images. In contrast, the S-CNN model

achieved an accuracy of 98.6% on unseen data, compared to only 42.3% for the F-CNN. The work also showed improved self-classification confidence using segmented images. The authors highlighted the significance of meaningful preprocessing and segmentation in improving the real-world applicability of deep learning-based plant disease detection systems.

[17] proposed an image-based plant leaf disease diagnosis and prediction framework using convolutional neural networks (CNNs), with a specific focus on tomato leaf diseases, utilising the PlantVillage dataset. The CNN architecture was designed to extract hierarchical features from leaf images and classify diseases effectively under controlled conditions. The proposed system achieved an overall classification accuracy exceeding 88%.

[18] employed convolutional neural networks to classify rice and potato leaf diseases, focusing on identifying four rice leaf diseases—bacterial blight, blast, brown spot, and tungro—and three potato leaf classes comprising healthy leaves, early blight, and late blight. A custom 16-layer CNN architecture was developed and trained using resized leaf images of dimension 128×128 pixels. The proposed CNN model achieved classification accuracies of 99.58% for rice leaf diseases and 97.66% for potato leaf diseases, outperforming traditional machine learning classifiers such as SVM, K-NN, Decision Tree, and Random Forest.

[19] proposed a hybrid computer vision and deep learning framework using a combination of Discrete Wavelet Transform (DWT), Principal Component Analysis (PCA), GLCM, and CNN classification for tomato leaf disease detection. The model was evaluated on a tomato leaf dataset containing six disease classes and healthy samples, achieving an overall accuracy of 99.09%, outperforming traditional approaches such as K-means clustering with SVM and PCA-based K-NN classifiers. Comparative analysis with existing methodologies demonstrated the superiority of the proposed feature-reduction and deep learning pipeline.

[20] proposed a lightweight CNN architecture with squeeze-and-excitation (SE) mechanisms that effectively capture inter-channel dependencies for classifying multiple plant leaves and diseases in the PlantVillage dataset. The model attained an accuracy of 99.79% with the lowest parameter count and smallest model size, when compared with other state-of-the-art models. The proposed model achieved superior feature discrimination due to the embedding of the SE block.

[10] surveyed convolutional neural network models extensively and reported that architectures such as VGG, ResNet, EfficientNet, GoogleNet, and MobileNet consistently achieved the highest classification accuracy across multiple plant disease datasets.

2.3. Transfer learning approaches for plant leaf disease classification

[2] used the EfficientNet deep-learning model to classify the 39 leaves available in the PlantVillage dataset. In an earlier work [21], the original dataset was augmented to improve the performance of deep neural networks, increasing the number of images from 55448 to 61486. The EfficientNet model was compared with other pre-trained models, including AlexNet, VGG16, ResNet50, and Inception V3. All the pre-trained models were trained on the ImageNet dataset, sped up by finetuning of pre-trained weights. Of the 8 models trained, the B4 and B5 models were superior to other CNN architectures. The B5 model achieved 99.91% accuracy in the original dataset, and the B4 model achieved 99.97% accuracy in the augmented dataset. In the EfficientNet architecture, even though

the input image size was necessarily resized to 132x132 due to hardware limitations, it yielded more successful results than other CNN models that received input images with higher resolutions. Another remarkable observation was that as the number of data points increased, the number of false positive predictions of models decreased, and precision increased.

[22] proposed a ResNet model that used deep-CNNs, along with image augmentation performed by the PyTorch library, to classify diseased tomato leaves within the PlantVillage dataset. Transfer learning was applied to fit the dataset without overfitting, using the feature extractor approach. The model yielded 94.6% accuracy before data augmentation and 97% accuracy after augmentation, proving the data augmentation approach successful. With an accuracy of 97%, the model could detect leaf diseases in a short period of time, but was limited by the high configuration hardware required for training.

In an advanced work of the detection of tomato leaf diseases in [23], the optimisation of hyperparameters was prioritised in the ResNet model to produce the highest possible classification accuracy on the PlantVillage dataset. At first, the Stochastic Gradient Descent (SGD) and Adaptive Moment Estimation (Adam) optimisers are compared between ResNet, AlexNet and GoogleNet models, with ResNet (SGD) yielding a 96.51% accuracy. Following this, the optimal batch size and number of iterations were found to be 16 and 4992, respectively, although the work suggests that the setting of batch size and number of iterations is dependent on the dataset and model used. At last, fine-tuning ResNet layers obtained the highest accuracy of 97.28%. The work suggested that moderate fine-tuning of layers would be the most practical application.

[24] evaluated deep CNN architectures like SqueezeNet and AlexNet models across the PlantVillage dataset. Although AlexNet achieved a slightly higher accuracy of 95.65%, SqueezeNet was nearly 80 times smaller and achieved comparable performance, making it suitable for real-time and resource-constrained environments. The work demonstrated that deep learning significantly improves classification performance while enabling practical field deployment.

[25] proposed a lightweight yet high-accuracy deep learning architecture, an uncertainty-aware MobileNetV3 framework for sugarcane leaf disease detection, which achieved a test accuracy of 97.23% with only 5.4 million parameters. Grad-CAM visualisations confirmed that the network focused on disease-relevant leaf regions, and real-time web deployment demonstrated its feasibility for precision agriculture.

[5] applied MobileNetV2 on the PlantVillage dataset, combined with Contrast Limited Adaptive Histogram Equalisation (CLAHE) to nullify the illumination problem found within the images, along with runtime data augmentation techniques to address class imbalance issues, and achieved an accuracy of 99.3% and a 99.12% F1 score, while having a model size of only 9.6 MB and 4.87 MFLOPS.

Overall, the reviewed literature demonstrated a clear evolution from classical machine learning models, which achieve moderate accuracies, toward deep learning and transfer learning architectures that consistently exceed 95% accuracy. Table 1 summarises the various works that have been surveyed. Recent research increasingly prioritised lightweight design, robustness, interpretability, and real-world deployment, indicating that modern plant and tomato leaf disease detection systems are moving beyond laboratory-scale classification toward practical agricultural solutions.

Table 1: Summary of related works on plant disease detection.

Reference	Method	Dataset	Accuracy (%)	Technique
[9]	Feature extraction with Random Forest	Kaggle dataset	73.38	ML
[7]	Colour analysis with MLP, K-NN & SVM	PlantVillage	86.45	ML
[17]	CNN	PlantVillage	88	DL
[15]	CNN	PlantVillage	91.2	DL
[13]	Segmentation with SVM	Self-collected dataset	93.75	ML
[24]	AlexNet	PlantVillage	95.65	DL
[8]	Colour channel analysis with CNN	Public Soyabean dataset	96	DL
[22]	ResNet	PlantVillage	97	DL
[25]	MobileNetV3	Open-source sugarcane leaf dataset	97.23	DL
[23]	Optimised ResNet	PlantVillage	97.28	DL
[16]	Segmentation with CNN	Mixed (Self-collected, PlantVillage & Internet)	98.6	DL
[19]	Feature extraction with CNN	Kaggle dataset	99.09	DL
[5]	MobileNetV2 with CLAHE	PlantVillage	99.3	DL
[18]	CNN	Publicly available rice dataset	99.58	DL
[20]	CNN with squeeze and excitation mechanism	PlantVillage	99.79	DL
[12]	Fuzzy Relevance Vector Machine	ICL leaf dataset	99.87	ML
[2]	EfficientNet B4	Augmented PlantVillage dataset	99.97	DL

3. Proposed Methodology

3.1. Dataset description

The tomato leaf dataset [11] had 546 images of diseased leaves and 482 images of healthy leaves, adding to a total of 1028 images. The images were shot in a Bangladeshi farm using mobile devices under normal environmental conditions. Figure 1 shows samples of the images collected by the dataset authors.

3.2. Dataset split

The dataset came pre-labelled and split into two directories: *train* contained 868 images, and *val* contained 160 images. The *val* directory was balanced with 80 healthy and 80 diseased leaves, but the *train* dataset had 402 healthy and 466 diseased leaves.

The following modifications were introduced in the dataset:

- The *val* directory was untouched. 100 images were used for validation, and 60 for testing in all the models.
- Image augmentation was applied to the *train* directory to create a total of 1000 training images, with balanced classes (500 healthy and 500 diseased images)

Following the changes, a dataset of 1160 images was acquired: 1000 for training and 160 for validation/testing with balanced class representation. Figures 2a and 2b visualise the dataset split accurately.



Figure 1: Sample images of healthy and diseased leaves.

3.3. Image augmentation

As mentioned above, image augmentation was performed on the training images to increase the number of images from 868 to 1000, creating 132 augmented images to ensure equal class distribution. Perfect class balance is not a prerequisite for effective machine learning design; however, it was intentionally maintained throughout this work.

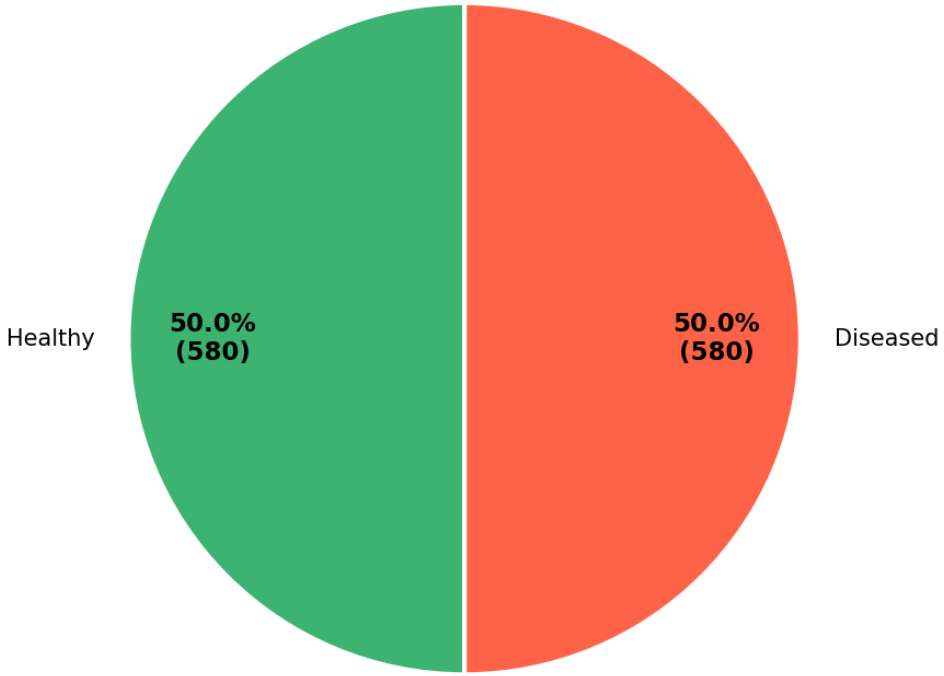
98 healthy and 34 diseased training images were chosen for augmentation. The following augmentation techniques were performed on the chosen image:

- Rotation - Rotated the image randomly within the range of [-20, 20] degrees with respect to the centre pixel.
- Shift - Moved all pixels of the image randomly within the range of [-30, 30] in both x and y directions.
- Zoom - Magnified/minified the image within the randomly chosen zoom factor of [0.7, 1.3]. The borders of the image were filled with black pixels upon minification and cropped to the original dimensions upon magnification.
- Flip - Mirrored the image along a randomly chosen axis: horizontal (1), vertical (0), or both axes simultaneously (-1).
- Rescale - Normalised pixel intensities to the full [0, 255] range by stretching the minimum and maximum values of the image.

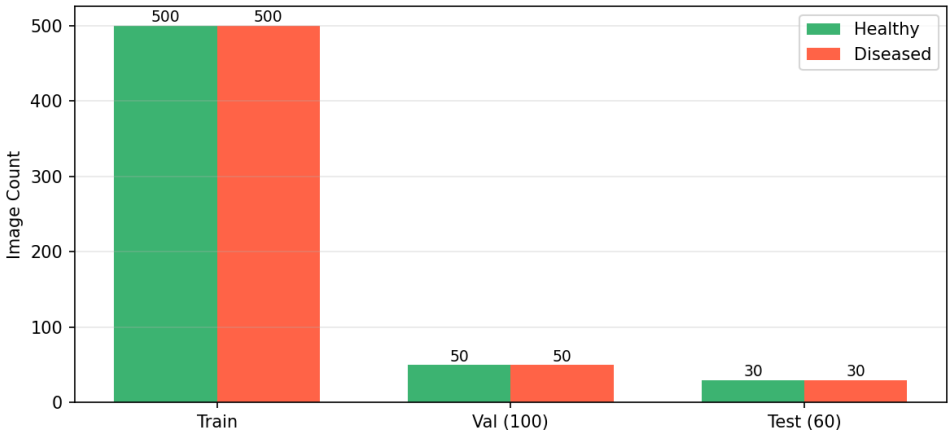
Figure 3 showcases the results of the image augmentation process.

3.4. Preprocessing

A comparative analysis of preprocessing techniques was conducted on the tomato leaf dataset to identify the approach yielding the highest classification accuracy.



(a) Class distribution of full dataset



(b) Dataset split

Figure 2: Results of the dataset split.



Figure 3: Augmented images (left to right): rotation, shift, zoom and flip

3.4.1. K-means clustering of colour channels

In this approach, the images were loaded in different colour channels across three colour spaces, using the NumPy and OpenCV packages from Python. The colour channels analysed were RGB (Red, Green, Blue), HSV (Hue, Saturation, Value) and the LAB colour space. For each channel, K-means clustering was applied on a sample of 60 images, targeting three clusters: the background of the image, the diseased part of the leaf and the healthy area of the leaf. The channel with the highest mean separation score (calculated through standard deviation) of all three clusters was ranked as the best colour channel for model training. Assuming the cluster with the second-highest intensity score forms the leaf mask, which was then refined using morphological operations (closing and opening) before being applied to isolate the leaf from the background. The colour channel analysis of 60 image samples revealed the best colour channel as saturation. Table 2 displays the scores of the different colour channels analysed. Figure 4 displays the results of the K-means segmentation.

Table 2: Summary of K-means segmentation.

Colour Channel	**Standard Deviation**
Saturation	56.2076
Red	49.9272
Blue	46.0907
Value	45.0536
L	43.6288
Green	42.3992
Hue	26.416
A	14.0815
B	13.4120

3.4.2. Contour segmentation

In this approach, each image was first converted to grayscale, reducing the input to brightness values alone and eliminating the additional complexity introduced by colour channels. A Gaussian blur was then applied to suppress pixel-level noise and smooth local

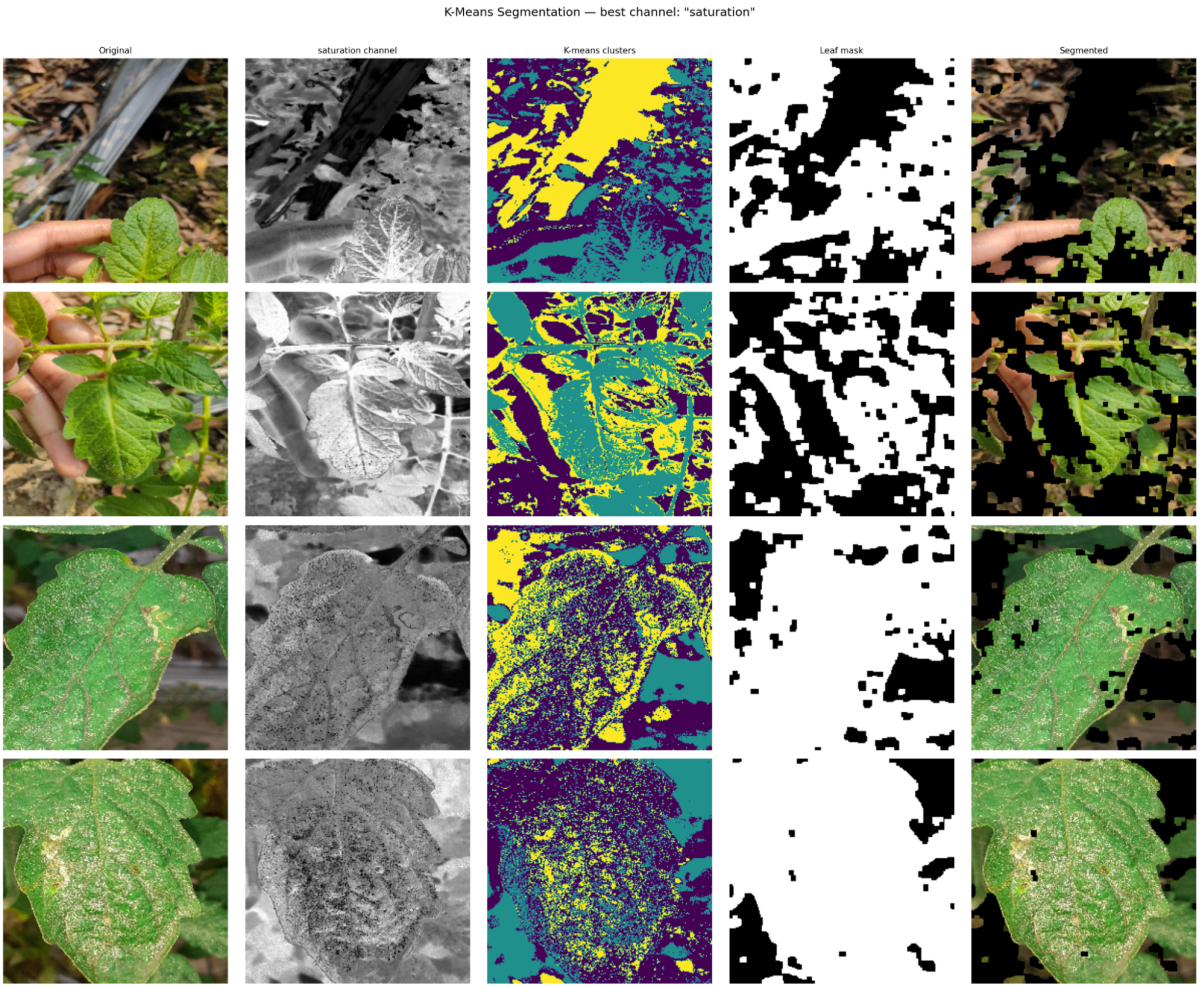


Figure 4: K-means segmentation of saturation colour channel

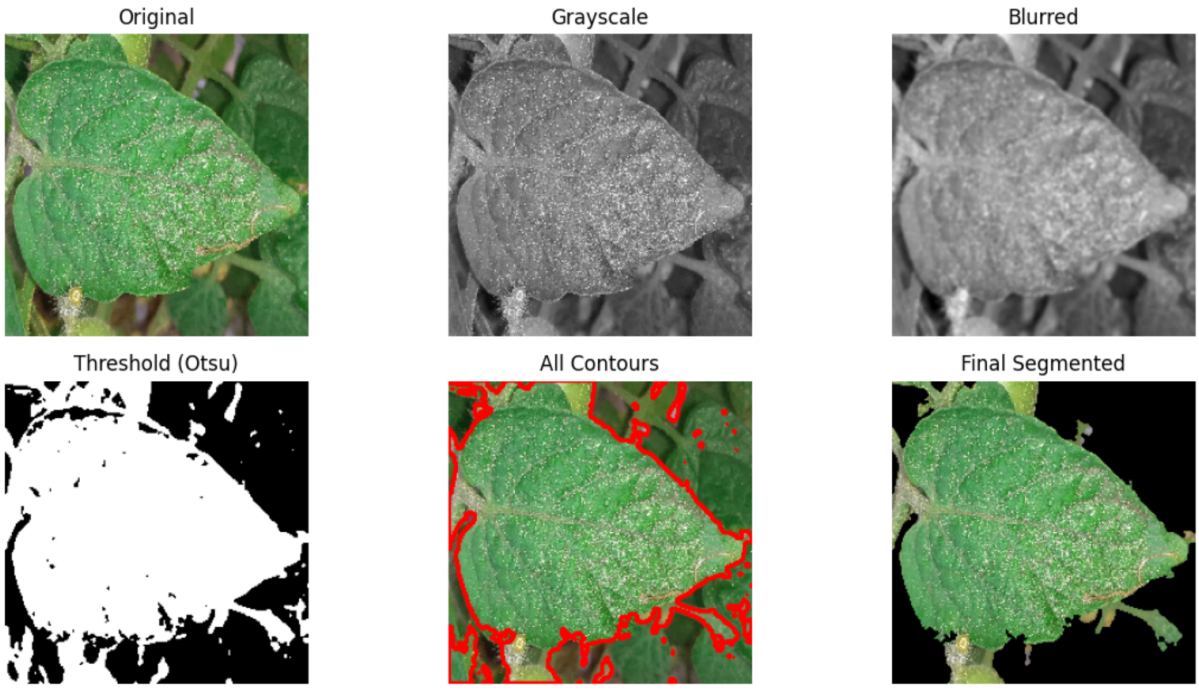


Figure 5: Contour segmentation visualisation

texture variations that would otherwise be incorrectly detected as edges in subsequent processing steps. Otsu's thresholding was applied to the blurred grayscale image to produce a binary representation, with the threshold value determined automatically from the image histogram rather than set manually, ensuring that pixels above the threshold were assigned to the foreground and all remaining pixels to the background.

Contour detection was then performed on the binary image using OpenCV, considering only external boundaries and disregarding any internal contours. From the set of detected contours, the largest by area was selected under the assumption that the leaf occupies the dominant region of the frame. A binary mask was constructed by filling this contour on a black background, and a bitwise AND operation was applied between the mask and the original image to retain only the pixels falling within the contour boundary, setting all exterior pixels to zero. In cases where no contours were detected, the original image was passed through unchanged to prevent pipeline failure, though such cases yield an unsegmented output.

Contour segmentation was computationally lightweight and performed reliably when the leaf was well-separated from the background. However, its dependence on the largest-contour assumption rendered it susceptible to failure in scenes containing multiple large objects, overlapping elements, or low contrast between the leaf surface and the background. Figure 5 presents the results of the contour segmentation preprocessing technique.

3.5. Model architectures and choice of hyperparameters

A unified deep learning pipeline was employed across nine models: CNN, SqueezeNet, VGG19, MobileNetV3, EfficientNetB0, ResNet50, InceptionV3, DenseNet201, and AlexNet across the two segmentation conditions, ensuring consistency in training and evaluation. All input images were resized to 224×224 resolution. For each architecture, a systematic hyperparameter search was conducted over a fixed candidate set, yielding 5 model variants

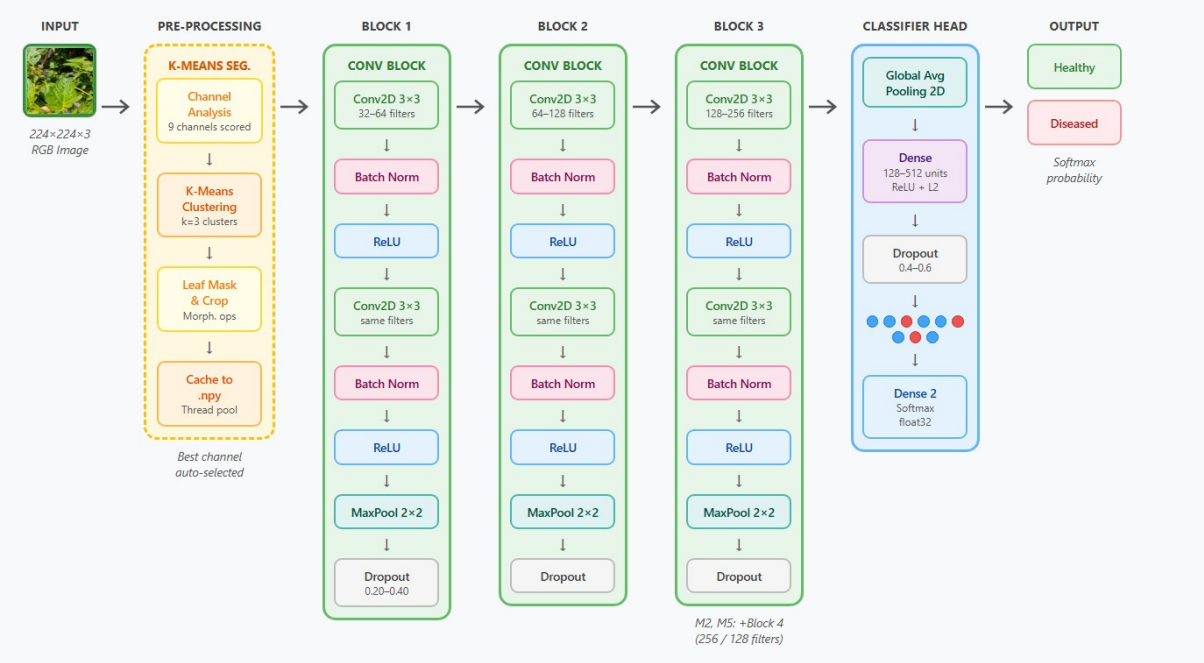


Figure 6: Architecture of the proposed CNN model

for the optimised CNN, DenseNet201, InceptionV3, SqueezeNet and AlexNet architectures, and 12 model variants for the VGG19, MobileNetV3, ResNet50 and EfficientNetB0 architectures. All models were trained on the augmented training set and selected based on validation accuracy on the 100-image validation split. The best-performing variant per architecture was then retrained on the combined train-plus-validation data and evaluated on the held-out test set.

3.5.1. CNN architecture

The optimised CNN was constructed using modular double-convolution blocks, where each block comprises two consecutive Conv2D–Batch Normalisation–ReLU layers followed by MaxPooling and Dropout. Multiple configurations were explored, including lightweight three-block models with filter sizes $[32, 64, 128]$, deeper four-block models with $[32, 64, 128, 256]$, and regularised variants with higher filter sizes $[64, 128, 256]$, increased dropout (0.40–0.60), and L2 regularisation ($\lambda = 10^{-3}$). These variants were trained using different optimisers, including Adam (learning rates 1×10^{-3}, 5×10^{-4}), RMSprop (5×10^{-4}), and SGD with Nesterov momentum (1×10^{-2}, momentum $= 0.9$), followed by Global Average Pooling and a dense classification head. Figure 6 displays the architecture of the optimised CNN model.

3.5.2. Transfer learning architectures

ImageNet pre-trained weights were loaded with the top classification layer removed, and a custom classification head was appended. This head consisted of a Global Average Pooling layer followed by one or two fully connected layers, optional Batch Normalisation, Dropout regularisation, and a sigmoid activation function for binary classification. Mixed-precision training was adopted for computational efficiency, while the final output layer was retained in float32 to maintain numerical stability. Optimisation was primarily performed

using Adam, with RMSprop and SGD with Nesterov momentum explored in selected configurations.

A consistent strategy was adopted with controlled fine-tuning. In the case of VGG19, MobileNetV3, EfficientNetB0, and ResNet50, the majority of the convolutional base was frozen, while the last 20 layers were unfrozen and fine-tuned using reduced learning rates. A grid search was conducted over classification head configurations, including dense layer sizes of 128 and 256 units, dropout rates of 0.3, 0.4, and 0.5, and learning rates of 1×10^{-3} and 1×10^{-4} using the Adam optimiser. For the deeper architectures InceptionV3 and DenseNet201, both frozen and fine-tuning strategies were explored. Three frozen-base variants were evaluated with different classification head depths and regularisation settings, along with two fine-tuning configurations in which the final 20 or 50 layers were unfrozen and trained using lower learning rates (1×10^{-4} and 5×10^{-5}).

The SqueezeNet-inspired model was implemented using Fire modules, each consisting of a squeeze layer with 1×1 convolutions followed by parallel expand layers with 1×1 and 3×3 filters, whose outputs are concatenated. Variants included lightweight four-module and deeper six-module configurations, along with regularised versions incorporating higher dropout and L2 penalties. These were trained using Adam (1×10^{-3}, 5×10^{-4}), RMSprop (5×10^{-4}), and SGD with Nesterov momentum (1×10^{-2}, momentum $= 0.9$).

Similarly, an AlexNet-inspired architecture was implemented from scratch, adapting the classical five convolutional layer design to the input resolution. Both lightweight (reduced filter count) and full-capacity configurations were evaluated, including a variant with a 4096-unit fully connected layer. Regularised versions with increased dropout and L2 penalties were also explored. Training configurations included Adam (1×10^{-3}, 5×10^{-4}), RMSprop (5×10^{-4}), and SGD with Nesterov momentum (1×10^{-2}, momentum $= 0.9$), with a reduced 1024-unit dense layer used in the SGD variant.

3.5.3. Hyperparameter tuning

Across all models, hyperparameter tuning focused on optimiser selection, learning rate variation (10^{-2} to 5×10^{-5}), dropout (0.3–0.6), L2 regularisation, and architectural depth. This consistent yet flexible methodology allowed for a robust comparison of lightweight, classical, and state-of-the-art deep learning architectures under a unified experimental framework.

3.6. Training protocol

All models were trained up to 40 epochs with a batch size of 32. Training was governed by `EarlyStopping` monitored on validation accuracy (patience = 20, minimum delta = 0.005, with best weights restored), and a `ReduceLROnPlateau` schedule monitored on validation loss (factor = 0.5, patience = 8, minimum learning rate = 1×10^{-6}). Model checkpointing was used to preserve the weights corresponding to the best observed validation accuracy. Following model selection, the best variant per architecture was retrained on the combined training and validation data and evaluated on the held-out test set. The algorithm detailing the pipeline is given below.

All experiments were conducted across two compute environments. The modified CNN, DenseNet201 and InceptionV3 variants were trained on a local workstation equipped with an NVIDIA RTX 3060 GPU (6 GB VRAM), while the transfer learning architectures, VGG19, MobileNetV3, EfficientNetB0, ResNet50, and AlexNet, were trained on a Google Cloud instance backed by an NVIDIA Tesla T4 GPU (16 GB VRAM) via Google

Colab. Mixed-precision training (`float16` compute, `float32` weights) was enabled in both environments to reduce memory consumption and accelerate tensor core utilisation, with output classification layers explicitly cast to `float32` to prevent softmax numerical instability.

Algorithm 1: Tomato Leaf Disease Classification Pipeline

Input: Tomato leaf disease images in I_{train} and I_{val} directories, model architecture A, segmentation condition S

Output: Predicted class label $\hat{y}$ and class probabilities y

Preprocess Images $P(I_{train}) \leftarrow$ Apply segmentation condition S to images in I_{train};
$P(I_{val}) \leftarrow$ Apply segmentation condition S to images in I_{val};
Split $P(I_{val})$ into validation set I_{val_split} and test set I_{test};
Model Variant Selection and Training Initialize model variants $M = \{m_1, m_2, \ldots, m_n\}$ of architecture A, each with distinct optimizer and learning rate configuration;
foreach *variant* m_k *in* M **do**
    Initialize m_k with pretrained weights and classification head;
    foreach *epoch* e **do**
        foreach *image* i *in* $P(I_{train})$ **do**
            Extract features $z_{CLS} \leftarrow m_k(i)$;
            Compute loss $\mathcal{L}$ and backpropagate; update parameters of m_k;
        end
        Evaluate m_k on I_{val_split}; record validation accuracy and loss;
        if *validation loss does not improve* **then**
            apply early stopping;
        end
    end
    Save best-performing checkpoint of m_k based on validation accuracy;
end
Best Variant Selection and Final Evaluation $m^{**} \leftarrow m_k$ with highest validation accuracy across all variants;
Retrain m^{**} on $P(I_{train}) \cup I_{val_split}$ with the same hyperparameter configuration;
Classification Compute class probabilities $y \leftarrow \text{softmax}(W_c \cdot z_{CLS} + b_c)$ for each image i in I_{test};
Predict label $\hat{y} \leftarrow \arg\max(y)$;
return $\hat{y}, y$;

3.7. Performance metrics

Model performance was assessed using accuracy, precision, recall, and F1-score computed on the 60-image held-out test set. Let TP, FP, and FN denote true positives, false positives, and false negatives, respectively. Precision, recall, and F1-score are defined as shown in Equations 1, 2 and 3, respectively:

$$Precision = \frac{\text{TP}}{(\text{TP} + \text{FP})} \tag{1}$$

$$Recall = \frac{\text{TP}}{(\text{TP} + \text{FN})} \tag{2}$$

$$F_1\ Score = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

4. Experimental Results & Discussion

Tables 3 and 4 summarise the test-set performance of the best-performing variant of each architecture under k-means clustering of colour channels and contour segmentation. Validation accuracies reflect the model-selection stage; test accuracies, precision, recall, and F1-score reflect final evaluation on the 60-image held-out set.

Table 3: Performance comparison of models on the test set under k-means clustering of colour channels.

Architecture	**Validation Accuracy**	**Test Accuracy**	**Precision**	**Recall**	**F1-Score**
AlexNet	0.83	0.6	0.6961	0.6	0.5442
SqueezeNet	0.967	0.75	0.83	0.75	0.73
EfficientNetB0	0.91	0.867	0.895	0.865	0.865
ResNet50	0.84	0.867	0.895	0.865	0.865
InceptionV3	0.89	0.867	0.87	0.87	0.87
DenseNet201	0.87	0.867	0.89	0.87	0.86
MobileNetV3	0.82	0.89	0.915	0.9	0.9
Optimised CNN	0.85	**0.983**	0.98	0.98	0.98
VGG19	0.85	**0.983**	0.985	0.985	0.98

Table 4: Performance comparison of models on the test set under contour segmentation.

Architecture	**Validation Accuracy**	**Test Accuracy**	**Precision**	**Recall**	**F1-Score**
AlexNet	0.78	0.757	0.845	0.775	0.763
SqueezeNet	0.77	0.77	0.84	0.77	0.763
InceptionV3	0.89	0.8167	0.82	0.82	0.82
MobileNetV3	0.8	0.85	0.86	0.85	0.85
DenseNet201	0.88	0.88	0.91	0.88	0.88
EfficientNetB0	0.87	0.917	0.905	0.9	0.9
VGG19	0.8899	0.93	0.935	0.935	0.935
ResNet50	0.87	0.93	0.94	0.935	0.935
Optimised CNN	0.92	0.95	0.95	0.95	0.95

4.1. K-means clustering of colour channels

Under K-means segmentation, the optimised CNN (RMSprop, lr $= 5 \times 10^{-4}$) and VGG19 jointly achieved the highest test accuracy of 98.33%, with F1-scores of 0.980 and 0.980,

respectively. VGG19 produced the highest precision and recall of any model in this condition at 0.985. MobileNetV3 was the next best-performing model at 90% test accuracy and an F1-score of 0.900. EfficientNetB0, ResNet50, InceptionV3, and DenseNet201 clustered closely in the 86–87% test accuracy range, with F1-scores between 0.860 and 0.870.

SqueezeNet exhibited a notable generalisation gap under this condition: its best variant achieved a validation accuracy of 96.67%, the highest recorded validation figure across the entire K-means group, yet test accuracy dropped to 75% with an F1-score of 0.733, suggesting that the selected variant overfitted to the validation distribution. AlexNet performed the weakest overall in this condition, reaching only 60% test accuracy and an F1-score of 0.544.

4.2. Contour segmentation

Under contour-based segmentation, performance across the board was more uniform than under K-means but generally lower at the top end. The optimised CNN again led the condition at 95% test accuracy and an F1-score of 0.950. VGG19 and ResNet50 tied at 93.33% test accuracy with matching F1-scores of 0.935. EfficientNetB0 followed at 91.67% test accuracy with an F1-score of 0.900. DenseNet201 recorded a test accuracy of 88% and the highest precision in this condition at 0.910.

InceptionV3 and SqueezeNet performed comparatively weakly under contour segmentation, with test accuracies of 81.67% and 77%, respectively. Notably, AlexNet showed a marked improvement over its K-means result in this condition, reaching 75.70% test accuracy with an F1-score of 0.763, and demonstrating higher precision (0.845) relative to its recall (0.775).

4.3. Ablation study

To assess the effect of the two segmentation techniques, an ablation study was performed to compare all the models' performance without image segmentation. Table 5 summarises the perfomance of the models under no segmentation condition.

The effect of segmentation was architecture-dependent. For VGG19 and the custom CNN, K-means segmentation was clearly beneficial, yielding the highest test accuracies observed for those architectures. For EfficientNetB0 and ResNet50, the no-segmentation baseline outperformed both segmented conditions by a margin of approximately 5 percentage points in F1-score, suggesting that the richer feature representations learned by deeper residual and efficient-scaling architectures may make explicit background removal redundant or even slightly detrimental. Other models which performed better than segmented variants include MobileNetV3 and DenseNet201. InceptionV3 and SqueezeNet performed most consistently under contour segmentation and K-means, respectively, though neither architecture produced competitive peak results relative to the leading models. AlexNet performed poorly across all segmentation conditions relative to the other models.

Table 5: Ablation study: Performance comparison of models on the test set with no segmentation.

Architecture	Validation Accuracy	Test Accuracy	Precision	Recall	F1-Score
AlexNet	0.8	0.76	0.63	0.6	0.58
InceptionV3	0.87	0.83	0.85	0.83	0.83
VGG19	0.89	0.85	0.885	0.85	0.845
SqueezeNet	0.93	0.867	0.873	0.867	0.867
DenseNet201	0.9	0.93	0.94	0.93	0.93
Optimised CNN	0.91	0.95	0.95	0.95	0.95
MobileNetV3	0.87	0.95	0.955	0.95	0.95
EfficientNetB0	0.95	0.967	0.97	0.965	0.97
ResNet50	0.95	0.967	0.97	0.965	0.97

4.4. Cross-condition analysis

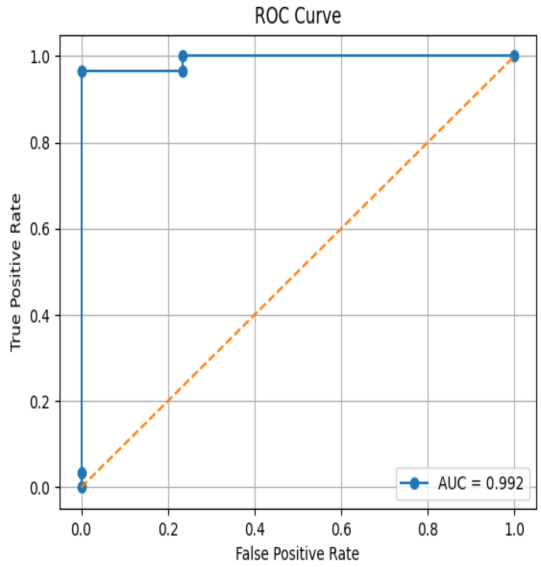
Across all 27 model–condition combinations, the five highest test accuracy values were: 98.33% (optimised CNN, K-means), 98.33% (VGG19, K-means), 96.67% (EfficientNetB0, no segmentation), 96.67% (ResNet50, no segmentation), and 95% (multiple models, multiple conditions). The optimised CNN and VGG19 therefore achieved the global best result in K-means segmentation, with 98.33% test accuracy and an F1-score of 0.980. ROC-AUC curves of these models are depicted in Figures 7a and 7b, loss curves in Figures 8a and 8b, and confusion matrices in Figures 9a and 9b.

A recurring pattern across architectures was that validation accuracy was not always a reliable predictor of test accuracy. SqueezeNet's K-means variant illustrates this most starkly: the highest validation accuracy in the K-means group (96.67%) corresponded to the lowest test accuracy among mid-tier models (75%). Conversely, the optimised CNN's K-means variant achieved its best test result (98%) despite a validation accuracy of only 85%, indicating that the combined train-plus-validation retraining step recovered meaningful generalisation capacity.

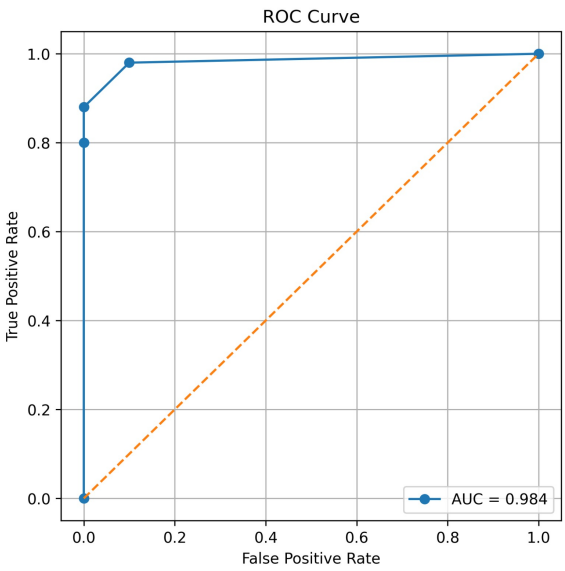
5. Conclusion & Future Work

This manuscript presented a systematic comparison of nine deep learning architectures across three image preprocessing conditions: K-means segmentation, contour-based segmentation, and no segmentation, for binary classification of tomato leaf disease. A total of 219 model variants were trained and evaluated under a consistent deep learning pipeline encompassing class-balanced data splitting, image augmentation, early stopping, and held-out test evaluation.

The primary finding is that the choice of architecture interacts significantly with the choice of preprocessing strategy, and that no single segmentation condition uniformly dominates across all models. The optimised CNN and VGG19 attained the highest test accuracy in the experiments at 98.33% under K-means segmentation, demonstrating that targeted background removal can provide a meaningful inductive bias for architectures whose feature extractors are either trained from scratch or frozen at a relatively shallow feature level. By contrast, EfficientNetB0 and ResNet50 achieved their peak performance



(a) Optimised CNN with K-means segmentation

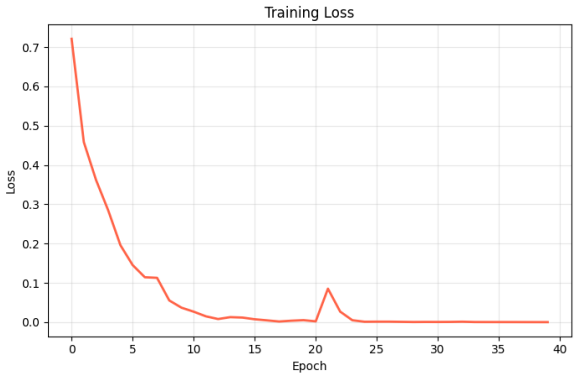


(b) VGG19 with K-means segmentation

Figure 7: ROC-AUC curves of the two best-performing models.

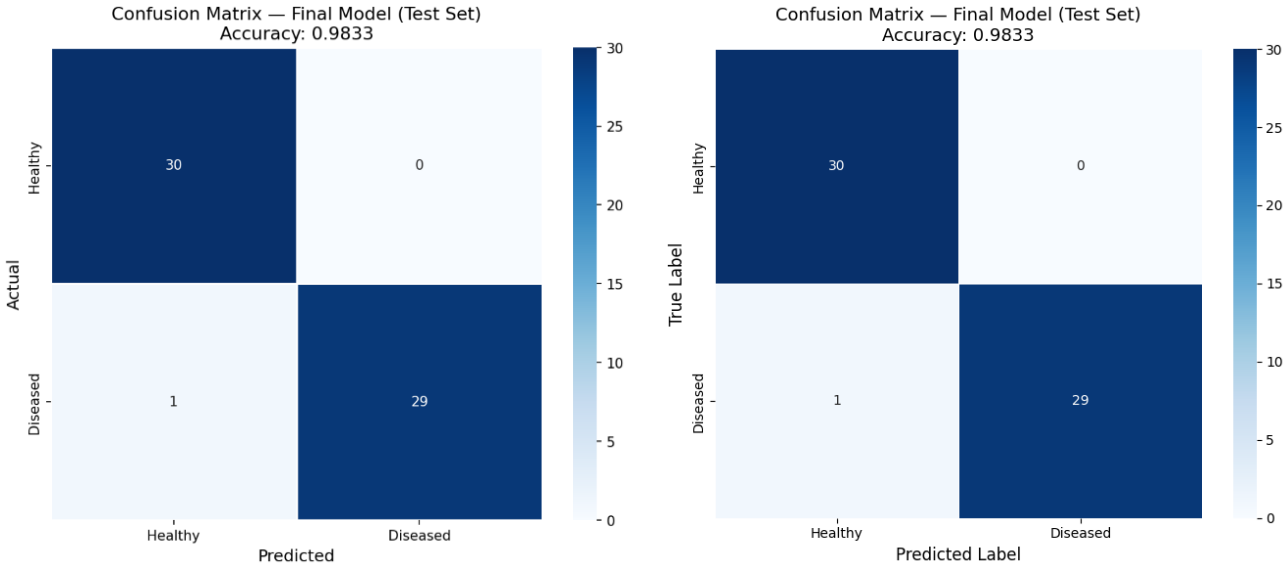


(a) Optimised CNN with K-means segmentation



(b) VGG19 with K-means segmentation

Figure 8: Loss curves of the two best-performing models.



(a) Optimised CNN with K-means segmentation

(b) VGG19 with K-means segmentation

Figure 9: Confusion matrices of the two best-performing models.

of 96.67% under no segmentation at all, highlighting that deep residual and compound-scaling architectures may already learn to attend to discriminative leaf regions without requiring explicit preprocessing.

The contour-based segmentation condition consistently produced moderate performances across most architectures, generally outperforming the no-segmentation baseline for lighter or less expressive models while falling short of K-means results for the architectures that benefited most from segmentation. This suggests that the quality of the leaf mask, and in particular the precision with which lesion regions are retained, matters more than the computational cost of the segmentation method itself.

SqueezeNet and AlexNet consistently underperformed relative to the other architectures across all conditions, with AlexNet recording the lowest F1-scores in two of the three conditions. The large generalisation gap observed for SqueezeNet under K-means segmentation, where an unusually high validation accuracy did not translate to test accuracy, points to a capacity-to-dataset mismatch that warrants further investigation with regularisation or ensemble strategies.

From a practical standpoint, the optimised CNN under K-means segmentation offers the most compelling result: it achieves 98.33% test accuracy with precision and recall of 0.980 using a comparatively lightweight architecture that does not depend on large pre-trained weights, making it well-suited for edge deployment scenarios. Where pre-trained transfer learning is preferred, VGG19 with K-means segmentation achieves an equivalent test accuracy with a marginally higher precision of 0.985. For deployments where preprocessing overhead must be minimised, EfficientNetB0 or ResNet50 without segmentation offer near-equivalent performance at 96.67% test accuracy.

Future work could explore multi-class disease classification beyond the binary healthy-versus-diseased setting, the integration of attention mechanisms to replace explicit segmentation, and the evaluation of model robustness under real-world acquisition conditions such as variable lighting, occlusion, and camera noise.

References

[1] Y. LeCun, Y. Bengio, and G. Hinton. "Deep learning". In: *Nature* 521.7553 (2015), pp. 436–444. DOI: 10.1038/nature14539.

[2] Ü. Atila, M. Uçar, K. Akyol, and E. Uçar. "Plant leaf disease classification using EfficientNet deep learning model". In: *Ecological Informatics* 61 (2021), p. 101182. DOI: 10.1016/j.ecoinf.2020.101182.

[3] O. Ahmed and A. Brifcani. "Gene expression classification based on deep learning". In: *Proceedings of the 4th Scientific International Conference Najaf (SICN)*. 2019, pp. 145–149. DOI: 10.1109/SICN47020.2019.9019357.

[4] R. Sharma. "Artificial intelligence in agriculture: a review". In: *2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS)*. IEEE, 2021, pp. 937–942. DOI: 10.1109/ICICCS51141.2021.9432187.

[5] Sabbir Ahmed, Md Bakhtiar Hasan, Tasnim Ahmed, Md Redwan Karim Sony, and Md Hasanul Kabir. "Less is more: Lighter and faster deep neural architecture for tomato leaf disease classification". In: *IEEE Access* 10 (2022), pp. 68868–68884. DOI: 10.1109/ACCESS.2022.3187203.

[6] L. Li, S. Zhang, and B. Wang. "Plant disease detection and classification by deep learning—a review". In: *IEEE Access* 9 (2021), pp. 56683–56698. DOI: 10.1109/ACCESS.2021.3069646.

[7] B. Luna-Benoso, J. C. Martínez-Perales, J. Cortés-Galicia, R. Flores-Carapia, and V. M. Silva-García. "Detection of diseases in tomato leaves by color analysis". In: *Electronics* 10.9 (2021), p. 1055. DOI: 10.3390/electronics10091055.

[8] B. V. Nikith, N. K. S. Keerthan, M. S. Praneeth, and T. Amrita. "Leaf disease detection and classification". In: *Procedia Computer Science* 218 (2023), pp. 291–300. DOI: 10.1016/j.procs.2023.01.011.

[9] N. Ganatra and A. Patel. "A multiclass plant leaf disease detection using image processing and machine learning techniques". In: *International Journal of Emerging Technologies* 11.2 (2020), pp. 1082–1086.

[10] T. D. Salka, M. B. Hanafi, S. M. Rahman, D. B. Zulperi, and Z. Omar. "Plant leaf disease detection and classification using convolution neural networks model: a review". In: *Artificial Intelligence Review* 58.10 (2025), p. 322. DOI: 10.1007/s10462-025-11234-6.

[11] M. K. Oni and T. T. Prama. "A comprehensive dataset of tomato leaf images for disease analysis in Bangladesh". In: *Data in Brief* 59 (2025), p. 111327. DOI: 10.1016/j.dib.2025.111327.

[12] B. VijayaLakshmi and V. Mohan. "Kernel-based PSO and FRVM: An automatic plant leaf type detection using texture, shape, and color features". In: *Computers and Electronics in Agriculture* 125 (2016), pp. 99–112. DOI: 10.1016/j.compag.2016.04.033.

[13] S. Vetal and R. S. Khule. "Tomato plant disease detection using image processing". In: *International Journal of Advanced Research in Computer and Communication Engineering* 6.6 (2017), pp. 293–297. DOI: 10.17148/IJARCCE.2017.6651.

[14] D. Hughes and M. Salathé. *An open access repository of images on plant health to enable the development of mobile disease diagnostics*. arXiv:1511.08060. 2015. DOI: 10.48550/arXiv.1511.08060.

[15] M. Agarwal, A. Singh, S. Arjaria, A. Sinha, and S. Gupta. "ToLeD: Tomato leaf disease detection using convolution neural network". In: *Procedia Computer Science* 167 (2020), pp. 293–301. DOI: 10.1016/j.procs.2020.03.225.

[16] P. Sharma, Y. P. S. Berwal, and W. Ghai. "Performance analysis of deep learning CNN models for disease detection in plants using image segmentation". In: *Information Processing in Agriculture* 7.4 (2020), pp. 566–574. DOI: 10.1016/j.inpa.2019.11.001.

[17] P. Goyal, D. K. Verma, and S. Kumar. "Diagnosis of plant leaf diseases using image based detection and prediction using machine learning approach". In: *Economic Computation and Economic Cybernetics Studies and Research* 57.4 (2023), pp. 293–312. DOI: 10.24818/18423264/57.4.23.18.

[18] R. Sharma, A. Singh, N. Z. Jhanjhi, M. Masud, E. S. Jaha, and S. Verma. "Plant disease diagnosis and image classification using deep learning". In: *Computers, Materials & Continua* 71.2 (2022), pp. 2125–2140. DOI: 10.32604/cmc.2022.020017.

[19] S. S. Harakannanavar, J. M. Rudagi, V. I. Puranikmath, A. Siddiqua, and R. Pramodhini. "Plant leaf disease detection using computer vision and machine learning algorithms". In: *Global Transitions Proceedings* 3.1 (2022), pp. 305–310. DOI: 10.1016/j.gltp.2022.03.016.

[20] S. Karthikeyan, R. Charan, S. Narayanan, and L. Jani Anbarasi. "Enhanced plant disease classification with attention-based convolutional neural network using squeeze and excitation mechanism". In: *Frontiers in Artificial Intelligence* 8 (2025), p. 1640549. DOI: 10.3389/frai.2025.1640549.

[21] G. Geetharamani and A. Pandian. "Identification of plant leaf diseases using a nine-layer deep convolutional neural network". In: *Computers & Electrical Engineering* 76 (2019), pp. 323–338. DOI: 10.1016/j.compeleceng.2019.04.011.

[22] M. Kaushik, P. Prakash, R. Ajay, and S. Veni. "Tomato leaf disease detection using convolutional neural network with data augmentation". In: *Proceedings of the 5th International Conference on Communication and Electronics Systems (ICCES)*. 2020, pp. 1125–1132. DOI: 10.1109/ICCES48766.2020.9138030.

[23] K. Zhang, Q. Wu, A. Liu, and X. Meng. "Can deep learning identify tomato leaf disease?" In: *Advances in Multimedia* 2018.1 (2018), p. 6710865. DOI: 10.1155/2018/6710865.

[24] H. Durmuş, E. O. Güneş, and M. Kırcı. "Disease detection on the leaves of the tomato plants by using deep learning". In: *Proceedings of the 6th International Conference on Agro-Geoinformatics*. 2017, pp. 1–5. DOI: 10.1109/Agro-Geoinformatics.2017.8047016.

[25] P. Pugazhendi, C. M. Badgujar, M. R. Ganapathy, and M. Arumugam. "Uncertainty-aware deep learning for sugarcane leaf disease detection using Monte Carlo dropout and MobileNetV3". In: *AgriEngineering* 8.1 (2026), p. 31. DOI: 10.3390/agriengineering8010031